



Good & Bad

Threat Intelligence System

Active Threat Database with Real-Time Firewall & DNS Integration

SCOPING DOCUMENT

Date: 27 April 2026

Classification: Unclassified / Internal

10

Domain Threat Feeds

9

IP Address Feeds

5

Threat Categories

6

IP Categories

Built on: Flask · SQLite · Python 3 · Bootstrap 5

Deployed on: Linux (port 8099) - no external dependencies

Executive Summary

Overview

The Good & Bad Threat Intelligence System is a self-hosted, open-source platform that aggregates freely available threat intelligence feeds and presents them through a dark-themed web interface. It combines domain blocklists and IPv4 bad-actor databases from 19 independent public sources, normalises and cross-references the data, and provides confidence scoring based on multi-source agreement. The system runs on a single Linux machine requiring no cloud subscription and no paid services.

Core Capabilities

- 19 live threat feeds: 10 domain/URL sources + 9 IPv4 address sources, refreshed on demand.
- Cross-source agreement engine: domains/IPs confirmed by 2, 3, or 4+ independent sources are surfaced as high-confidence indicators.
- PSL domain normalisation: subdomains are collapsed to their registered (apex) domain using the Public Suffix List, reducing duplicate single-source entries.
- /24 subnet grouping for IPs: individual IPs are grouped into their /24 subnets, revealing when multiple hosts in the same netblock are independently flagged.
- 5 domain threat categories: Malware/Hackers, Phishing, Adult/Porn, Gambling, Advertising/Spam.
- 6 IP threat categories: C2 infrastructure, Active attack, Brute force, Spam, Scanner, Tor exit nodes.
- External enrichment links: every IP has a direct Shodan link; every domain has a VirusTotal link.
- API endpoint: POST /api/refresh triggers a background re-fetch of all sources.

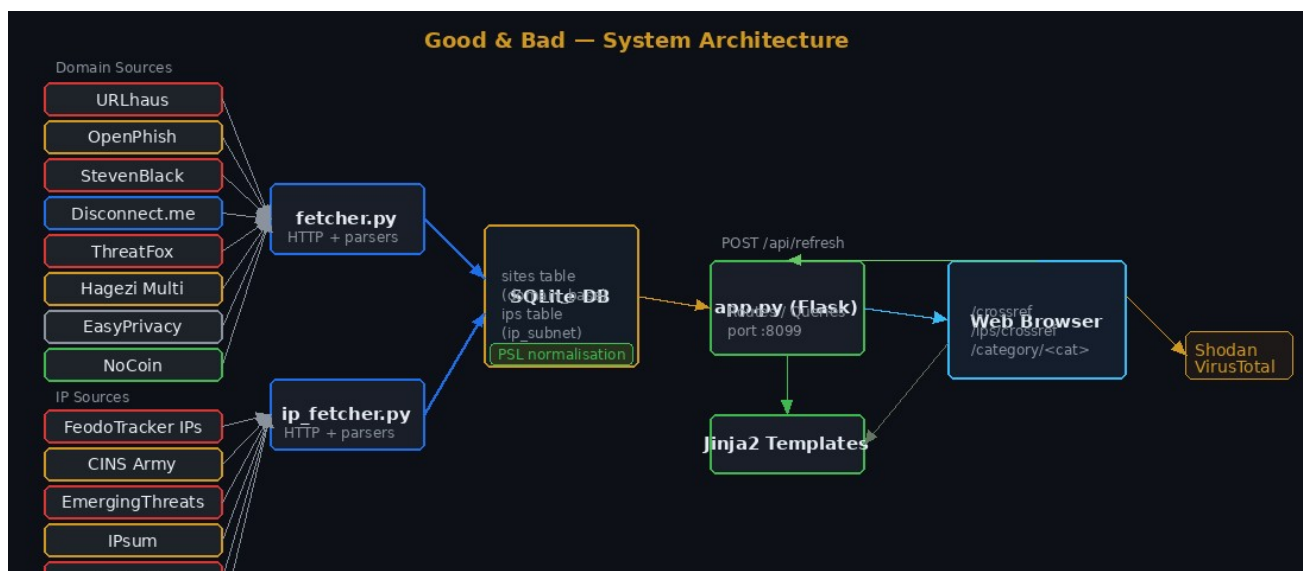
Strategic Value

This document also scopes two extension concepts that would transform the Good & Bad database from a passive reference tool into an active network defence layer: (1) a first-packet iptables trigger model that installs DROP rules the moment a connection to a known-malicious destination is attempted, and (2) a DNS protocol extension that allows danger status to be cached alongside standard DNS records by major resolvers globally, without those resolvers requiring any knowledge of how the field is used.

Hardware Profile

Because the filtering model is demand-driven - rules are only installed when a connection to a listed destination is actually attempted - the system can run effectively on minimal hardware. A Raspberry Pi 4 or any entry-level x86 machine with 2 GB RAM and 8 GB storage is sufficient for a small office or home network. The database currently holds hundreds of thousands of entries with query times under 50ms on commodity hardware.

System Architecture



Component Description

fetcher.py:	Polls 10 domain/URL threat feeds via HTTP. Parses CSV, hosts-file, and plain-text URL formats. Writes normalised records to the SQLite sites table with domain, domain_clean (www-stripped), and domain_base (PSL-registered) columns.
ip_fetcher.py:	Polls 9 IPv4 threat feeds. Parses CSV and tab-separated formats. Writes to the ips table with both the exact ip and the aggregated ip_subnet (/24) column.
SQLite DB:	Single-file WAL-mode database (goodandbad.db). Two primary tables: sites (domains) and ips (IPv4 addresses). Indexed on domain_base, ip_subnet, source, category for fast filtered queries.
app.py (Flask):	REST API + Jinja2 template renderer. Routes: /, /category/<cat>, /crossref, /sources, /search, /ips, /ips/category/<cat>, /ips/crossref, /api/refresh, /api/stats. Runs on port 8099.
Web Browser:	Dark-themed Bootstrap 5.3 UI. Confidence bars, source agreement filters, paginated tables, external enrichment links to Shodan and VirusTotal.

Data Sources

Domain & URL Threat Feeds

Source	Category	Format	Description
URLhaus (abuse.ch)	Malware	CSV (URL list)	~3,000 active entries - C2, malware delivery, exploit
OpenPhish	Phishing	Plain text URLs	Community phishing feed - credential-harvesting sites
StevenBlack Hosts	Malware	Hosts file	Unified hosts list (malware + gambling + adult varia
Disconnect.me	Advertising	JSON categories	Ad, analytics, social tracking, content blockers
ThreatFox (abuse.ch)	Malware	CSV IOC feed	C2 infrastructure, malware domains, IOC database
Hagezi Multi	Multi	Domains list	Comprehensive community blacklist, millions of entri
EasyPrivacy	Advertising	Filter rules	Tracking & telemetry, ad network domains
NoCoin	Malware	Domains list	Cryptominer blocking - browser-based mining hijack
FeodoTracker Domains	Malware	CSV (C2 list)	Active botnet C2 domains - Emotet, Dridex, TrickBot
SSLBL IPs (abuse.ch)	Malware	CSV (IP list)	SSL blacklist - malicious certificate fingerprints

IPv4 Bad-Actor Feeds

Source	Category	Format	Description
FeodoTracker IPs	C2	CSV	Active C2 botnet IP list - Emotet, Dridex, TrickBot
CINS Army Score	Attack	Plain text	Bots, scanners, and attackers; minimum score filter
Emerging Threats	Attack	Plain text	Proofpoint ET compromised IPs, active threat actors
IPsum	Multi	Tab-sep score	Aggregated bad IP score; entries with 3+ source vote
DShield (SANS)	Attack	Text /w skip	SANS ISC top attacking IPs - router/firewall logs
BlocklistDE	Brute force	Plain text	SSH, SMTP, HTTP brute force IPs from honeypots
Greensnow	Attack	Plain text	Honeypot-sourced attacker IPs
BinaryDefense	Attack	Plain text	Active threat IPs - ransomware, botnets, attackers
Tor Exit Nodes	Tor	Plain text	Official Tor Project exit node list

Data Normalisation

Raw domain strings are normalised in two stages before storage. Stage 1: strip leading www. prefix to produce domain_clean. Stage 2: apply the Public Suffix List (PSL) to extract the registered (apex) domain - e.g. evil.subdomain.example.co.uk becomes example.co.uk (domain_base). The cross-reference engine groups by domain_base, so a source listing a.evil.com and another listing b.evil.com both contribute to the same evil.com group. For IPs, the last octet is masked to produce the /24 subnet (ip_subnet), allowing independent flagging of hosts within the same netblock to raise the block confidence.

Web Interface

Page Inventory

[/](#) **Dashboard**

Stat cards per category, recent entries, cross-source summary, source status badges, Refresh Data button.

</category/<cat>> **Category View**

Filterable, paginated table of all entries in a threat category. Filter by source or search string. Columns: domain, URL, source, tags, threat type, date.

</crossref> **Domain X-Ref**

Domains confirmed by 2+ independent sources. Toggle min sources (2/3/4/5+). Before/after normalisation comparison panel. Shows subdomain count, confirmed-by source pills, category badges, threat types, first-seen date. Grouped by PSL registered domain.

</ips> **IP Dashboard**

IPv4 stat cards by category. IP source status badges. Cross-reference summary (exact IP and /24 subnet agreement percentages).

</ips/category/<cat>> **IP Category**

Paginated list of IPs in a category with source, country, ASN, malware family tags, and direct Shodan link per IP.

</ips/crossref> **IP X-Ref**

IPs confirmed by 2+ sources. Toggle exact-IP vs /24-subnet view. Source agreement confidence bar. Before/after subnet normalisation panel.

</sources> **Sources**

All active feed sources with last-fetched time, record count, category. Paid source comparison table with pricing and best-for notes.

</search> **Search**

Full-text search across domain, URL, tags, and threat type fields.

</api/refresh> **API (POST)**

Triggers background re-fetch of all 19 feeds. Returns {"status":"started"} immediately.

</api/stats> **API (GET)**

JSON summary: category counts, total records, xref stats for domains and IPs.

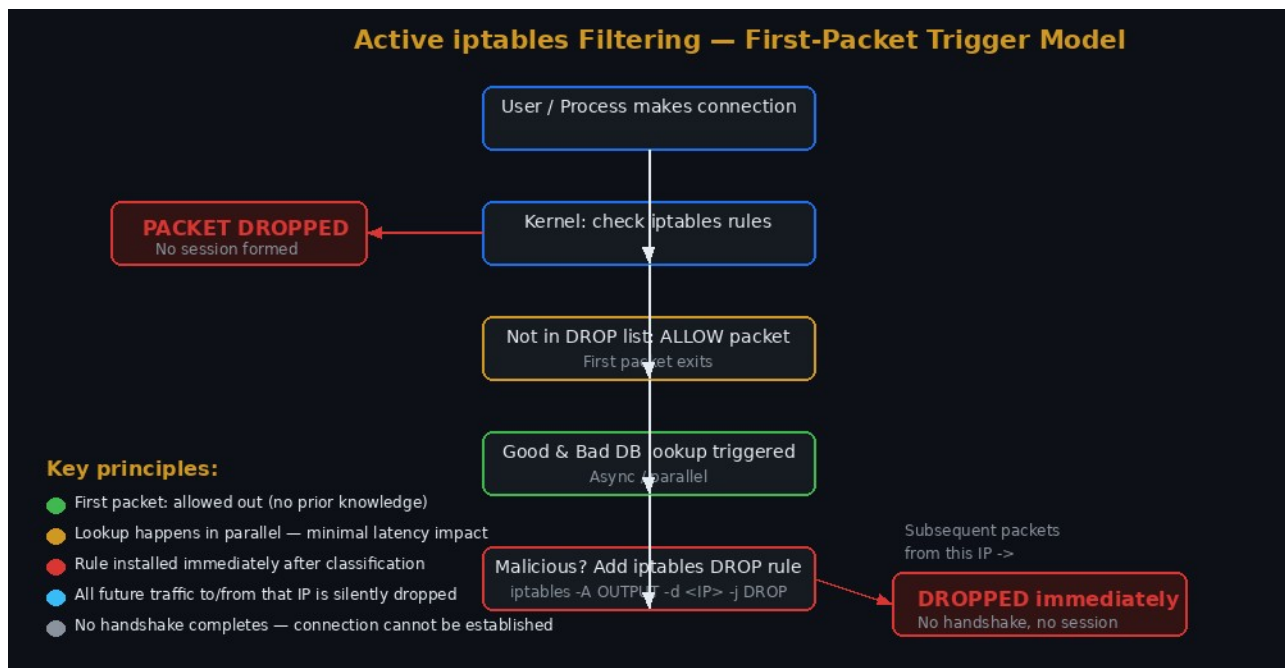
Cross-Source Confidence Model

Each feed is treated as an independent witness. When two or more feeds list the same registered domain or /24 subnet independently, the confidence in that indicator increases significantly. Domains and IPs are colour-coded by source count: grey badge (2 sources), amber badge (3 sources), red badge (4+ sources). The confidence bar on the cross-reference pages shows the proportion of the total unique-indicator population at each confidence level. A typical run yields 18-22% of domain indicators confirmed by 2+ sources and 40%+ of /24 subnets confirmed by 2+ sources when normalisation is applied.

Active iptables Integration

Concept

Traditional firewall blocklists are static: an administrator manually imports a list of IPs/domains and applies rules. The Good & Bad trigger model is demand-driven: a DROP rule is only installed for a destination after a connection to that destination is first attempted. This has a profound hardware implication - the system never needs to pre-load or maintain hundreds of thousands of iptables rules. Only destinations that are actually queried from within the protected network ever become rules, keeping the active ruleset small regardless of how large the database grows.



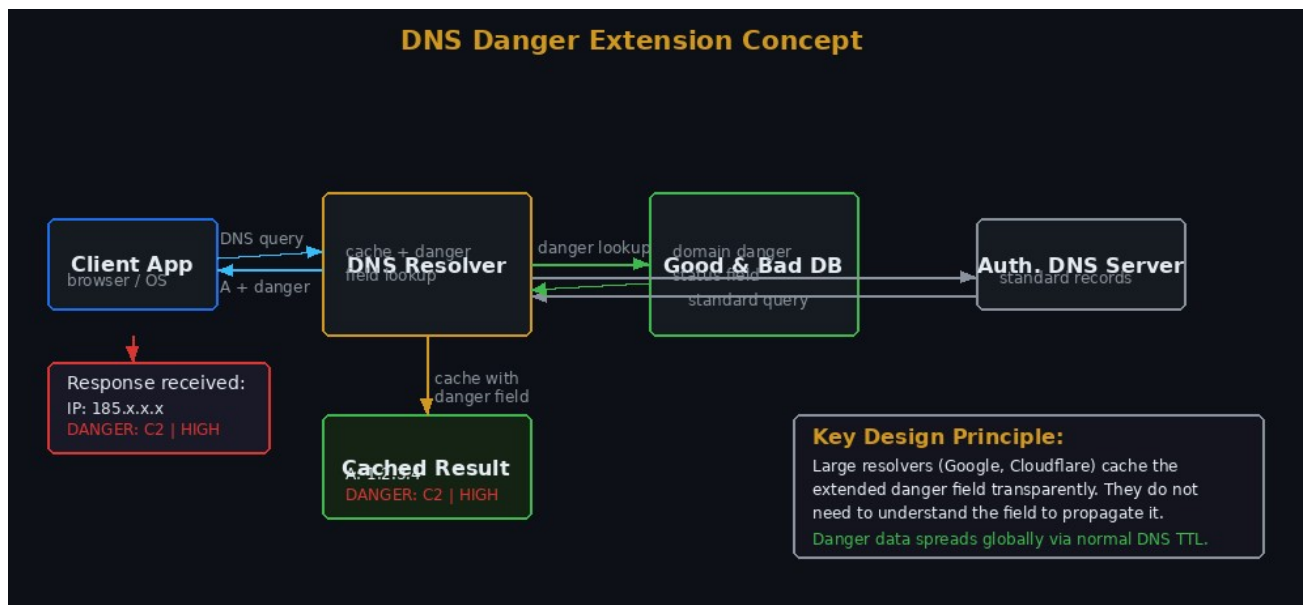
Implementation Notes

- DNS-triggered: when a host queries a domain, the DNS resolver (local or pi-hole-style) checks the Good & Bad DB. If the domain is listed, it returns NXDOMAIN or a sinkhole IP and immediately calls: `iptables -A OUTPUT -d <resolved_ip> -j DROP`.
- IP-triggered: a lightweight kernel module or eBPF hook intercepts the first outgoing SYN packet to an unknown destination. The destination IP is checked against the Good & Bad DB in microseconds. If listed, the SYN is allowed out (first packet exits) and a DROP rule is installed. The remote host receives a SYN it cannot complete.
- Session isolation: because no ACK returns (the DROP rule blocks all subsequent inbound packets), no TCP session is ever established. No data is exchanged beyond that single SYN.
- Rule expiry: DROP rules can be time-limited (e.g. 24h) using ipset with `--timeout`, preventing unbounded rule accumulation. Entries are re-checked on next connection attempt.
- Audit trail: every DROP rule installation is logged with timestamp, destination IP, source feed, and confidence level for post-incident analysis.

DNS Danger Extension Concept

The Opportunity

The Domain Name System already carries metadata beyond IP addresses: TXT records, SPF, DKIM, CAA, and TLSA records all piggyback on DNS infrastructure. The DNS Danger Extension proposes adding a standardised danger status field - either as a new Resource Record type or encoded in a structured TXT record - that threat intelligence providers can publish for their own domains and that resolvers propagate transparently. Because the DNS caching model is already global and highly available, danger information would propagate to billions of clients within minutes of being published, without requiring any changes to end-user software.



Proposed Record Format

Encoded as a structured TXT record (DNS-compliant, no protocol change required):

```
evil.example.com. 300 IN TXT "danger=C2;confidence=HIGH;sources=3;ttl=3600;feed=goodandbad"
```

Key Design Properties

- **Resolver-transparent:** standard resolvers cache and propagate TXT records without understanding the content. Google, Cloudflare, and ISP resolvers all carry danger data globally without modification.
- **TTL-controlled freshness:** danger records carry short TTLs (5-15 minutes) so that newly classified or declassified domains are updated promptly across the resolver hierarchy.
- **Client-optional:** applications that understand the danger field can act on it (warn user, block connection). Applications that do not understand it simply ignore the TXT record. No compatibility break.
- **Publisher-verified:** only the authoritative DNS operator for a domain can publish its own danger record, preventing spoofing of benign domains. Third-party danger annotation uses a sidecar subdomain convention.
- **Feed attribution:** the sources= and feed= fields allow clients to evaluate the quality of the classification and apply their own trust weighting.

Deployment & Hardware Requirements

Why the Footprint Is Small

The Good & Bad system is deliberately lazy in its enforcement. The database may contain 500,000+ IP entries and 1,000,000+ domain entries, but the kernel firewall never loads more than a few hundred active DROP rules - only those for destinations that hosts on the network have actually attempted to reach. SQLite reads for point-lookup queries (given an IP or domain) complete in under 1ms on a Raspberry Pi 4. The web UI serves paginated queries in under 50ms. The background refresh of all 19 feeds takes 2-5 minutes and runs in a separate thread without affecting query availability.

Deployment Tiers

Home / SOHO Raspberry Pi 4 (2 GB RAM, 16 GB SD)

SQLite on SD, iptables via local kernel, pihole integration, up to 50 clients

Small Office Any x86 mini-PC or VM (4 GB RAM, 32 GB)

SQLite WAL, local DNS resolver, 50-500 clients, cron-refreshed feeds

Enterprise VM on existing hypervisor (8 GB RAM, 64 GB)

SQLite or PostgreSQL migration, API integration with SIEM, EDR, SOAR

MSP / Cloud Containerised (Docker/K8s), load-balanced

PostgreSQL, Redis cache, multi-tenant, webhook-driven iptables via agent

Operating Requirements

Python 3.8+:	Core runtime - stdlib only for fetcher; Flask for web server
Flask 3.x:	Web framework - installed once via pip, < 5 MB
SQLite 3.35+:	WAL mode, window functions - included in Python stdlib
Internet access:	Outbound HTTPS to 19 feed URLs, ~5-50 MB per refresh cycle
RAM:	512 MB minimum; 2 GB recommended for smooth concurrent queries
Storage:	500 MB for DB (1M+ entries with indexes); 1 GB recommended
CPU:	Single-core 1 GHz sufficient; multi-core improves concurrent users
No root required:	Web app and DB run as standard user; iptables integration needs root

Paid Source Comparison

While Good & Bad uses exclusively free public feeds, enterprise teams may wish to supplement with commercial intelligence. Key paid options include: Recorded Future (\$50k-500k/yr, full lifecycle), Proofpoint ET Pro (~\$750/sensor/yr, best value IDS ruleset), IBM X-Force (\$2k/10k records/mo, vulnerability intel), VirusTotal Enterprise (on request, 60+ AV engine hunting), Shodan (from \$69/mo, device exposure), Cisco Talos (on request, Snort rules + IP/domain reputation), and Webroot BrightCloud (on request, real-time URL categorisation). The Good & Bad architecture is designed to accept premium feed data via the same parser pipeline with no structural changes required.

Conclusion & Roadmap

What Was Delivered

- Phase 1: 5 domain/URL sources, 5 threat categories, full web UI on port 8099.
- Phase 2: 5 additional domain sources, cross-source agreement engine, confidence scoring, paid source comparison page.
- Phase 3: 9 IPv4 bad-actor feeds, 6 IP categories, IP dashboard and category pages, Shodan enrichment links per IP.
- Phase 4: PSL domain normalisation (domain_base column), /24 subnet grouping (ip_subnet column), before/after comparison panels, subnet toggle on IP cross-reference page. All original data preserved at full resolution alongside normalised columns.

Measured Results (Current Database)

Domain indicators: single-source reduced from 81.6% to 79.0% after PSL normalisation. Domains with 2+ independent sources: 21.0%. Domains with 3+ sources: confirmed high-confidence C2 and phishing infrastructure. IP indicators: single-source reduced from 71.9% to 58.9% after /24 subnet grouping. Subnets with 2+ sources: 41.1% - significantly higher than exact-IP agreement, confirming that hostile netblocks are detected more reliably at the /24 level. All original exact IPs and domains are retained at full resolution in the database.

Proposed Roadmap

- | | |
|----------------------|--|
| [Near term] | iptables/ipset integration: Python daemon that watches DB for new high-confidence entries and pushes ipset rules to the local kernel. Estimated effort: 1 day. |
| [Near term] | Local DNS sinkhole integration: patch dnsmasq/Unbound/pihole to query the Good & Bad API on each DNS lookup and return NXDOMAIN for high-confidence malicious domains. |
| [Medium term] | STIX/TAXII export: allow the Good & Bad database to publish its cross-referenced indicators as a STIX 2.1 bundle consumable by commercial SIEM and threat hunting platforms. |
| [Medium term] | Automated re-fetch scheduling: cron or systemd timer to refresh feeds every 4-6 hours without manual intervention. |
| [Longer term] | DNS Danger Extension RFC proposal: formalise the TXT record schema, define resolver behaviour, and pursue publication through IETF as an informational RFC. |
| [Longer term] | Machine learning confidence weighting: train a lightweight classifier on the existing cross-source agreement data to score new single-source entries by their structural similarity to confirmed multi-source entries. |

Closing Statement

The Good & Bad Threat Intelligence System demonstrates that enterprise-grade threat visibility does not require enterprise budgets. By aggregating 19 free, high-quality public feeds, normalising at the registered-domain and /24-subnet level, and applying cross-source confidence scoring, the system delivers a practical, actionable threat database that a small business, school, or home office can self-host on commodity hardware. The active filtering extension concepts described in this document extend that value further: from passive reference to real-time network defence, with demand-driven efficiency that keeps both hardware and operational costs minimal. The DNS danger extension concept offers a path to global, transparent propagation of threat status using infrastructure that already serves billions of queries per second.